

XCS234 Assignment 3

Due Sunday, September 28 at 11:59pm PT.

Guidelines

1. If you have a question about this homework, we encourage you to post your question on our Slack channel, at <http://xcs234-scpd.slack.com/>
2. Familiarize yourself with the collaboration and honor code policy before starting work.
3. For the coding problems, you must use the packages specified in the provided environment description. Since the autograder uses this environment, we will not be able to grade any submissions which import unexpected libraries.

Submission Instructions

Written Submission: Some questions in this assignment require a written response. For these questions, you should submit a PDF with your solutions online in the online student portal. As long as the PDF is legible and organized, the course staff has no preference between a handwritten and a typeset L^AT_EX submission. If you wish to typeset your submission and are new to L^AT_EX, you can get started with the following:

- Type responses only in `submission.tex`.
- Submit the compiled PDF, **not** `submission.tex`.
- Use the commented instructions within the `Makefile` and `README.md` to get started.

Coding Submission: Some questions in this assignment require a coding response. For these questions, you should submit only the `src/submission.py` file in the online student portal. For further details, see Writing Code and Running the Autograder below.

Honor code

We strongly encourage students to form study groups. Students may discuss and work on homework problems in groups. However, each student must write down the solutions independently, and without referring to written notes from the joint session. In other words, each student must understand the solution well enough in order to reconstruct it by him/herself. In addition, each student should write on the problem set the set of people with whom s/he collaborated. Further, because we occasionally reuse problem set questions from previous years, we expect students not to copy, refer to, or look at the solutions in preparing their answers. It is an honor code violation to intentionally refer to a previous year's solutions. For SCPD classes, it is also important that students avoid opening pull requests containing their solution code on the shared assignment repositories. More information regarding the Stanford honor code can be found at <https://communitystandards.stanford.edu/policies-and-guidance/honor-code>.

Writing Code and Running the Autograder

All your code should be entered into `src/submission.py`. When editing these files, please only make changes between the lines containing `### START_CODE_HERE ###` and `### END_CODE_HERE ###`. Do NOT make changes to files other than `src/submission.py`.

The unit tests in `src/grader.py` (the autograder) will be used to verify a correct submission. Run the autograder locally using the following terminal command within the `src/` subdirectory:

```
$ python grader.py
```

There are two types of unit tests used by the autograder:

- **basic:** These tests are provided to make sure that your inputs and outputs are on the right track, and can be run locally.

- **hidden:** These unit tests are NOT visible locally. These hidden tests will be run when you submit your code to the Gradescope autograder via the online student portal, and will provide feedback on how many points you have earned. These tests will evaluate elements of the assignment, and run your code with more complex inputs and corner cases. Just because your code passed the basic local tests does not necessarily mean that they will pass all of the hidden tests.

For debugging purposes, you can run a single unit test locally. For example, you can run the test case `3a-0-basic` using the following terminal command within the `src/` subdirectory:

```
$ python grader.py 3a-0-basic
```

Before beginning this course, please walk through the [Anaconda Setup for XCS Courses](#) to familiarize yourself with the coding environment. Use the env defined in `src/environment.yml` to run your code. This is the same environment used by the online autograder.

Test Cases

The autograder is a thin wrapper over the python `unittest` framework. It can be run either locally (on your computer) or remotely (on SCPD servers). The following description demonstrates what test results will look like for both local and remote execution. For the sake of example, we will consider two generic tests: `1a-0-basic` and `1a-1-hidden`.

Local Execution - Basic Tests

When a basic test like `1a-0-basic` passes locally, the autograder will indicate success:

```
----- START 1a-0-basic: Basic test case.
----- END 1a-0-basic [took 0:00:00.000062 (max allowed 1 seconds), 2/2 points]
```

When a basic test like `1a-0-basic` fails locally, the error is printed to the terminal, along with a stack trace indicating where the error occurred:

```
----- START 1a-0-basic: Basic test case.
<class 'AssertionError'>
{'a': 2, 'b': 1} != None ← This error caused the test to fail.
File "/Users/grinch/Local_Documents/Software/anaconda3/envs/XCS221/lib/python3.6/unittest/case.py", line 59, in testPartExecutor
    yield
File "/Users/grinch/Local_Documents/Software/anaconda3/envs/XCS221/lib/python3.6/unittest/case.py", line 605, in run
    testMethod()
File "/Users/grinch/Local_Documents/SCPD/XCS221/A1/src/graderUtil.py", line 54, in wrapper
    result = func(*args, **kwargs)
File "/Users/grinch/Local_Documents/SCPD/XCS221/A1/src/graderUtil.py", line 83, in wrapper
    result = func(*args, **kwargs)
File "/Users/grinch/Local_Documents/SCPD/XCS221/A1/src/grader.py", line 23, in test_0
    submission.extractWordFeatures("a b a") ← In this case, start your debugging
                                           in line 23 of grader.py.
File "/Users/grinch/Local_Documents/Software/anaconda3/envs/XCS221/lib/python3.6/unittest/case.py", line 829, in assertEquals
    assertion_func(first, second, msg=msg)
File "/Users/grinch/Local_Documents/Software/anaconda3/envs/XCS221/lib/python3.6/unittest/case.py", line 822, in _baseAssertEqual
    raise self.failureException(msg)
----- END 1a-0-basic [took 0:00:00.003809 (max allowed 1 seconds), 0/2 points]
```

Remote Execution

Basic and hidden tests are treated the same by the remote autograder, however the output of hidden tests will only appear once you upload your code to GradeScope. Here are screenshots of failed basic and hidden tests. Notice that the same information (error and stack trace) is provided as the in local autograder, now for both basic and hidden tests.

1a-0-basic) Basic test case. (0.0/2.0)

```
<class 'AssertionError': {'a': 2, 'b': 1} != None
File "/autograder/source/miniconda/envs/XCS221/lib/python3.6/unittest/case.py", line 59, in testPartExecutor
    yield
File "/autograder/source/miniconda/envs/XCS221/lib/python3.6/unittest/case.py", line 605, in run
    testMethod()
File "/autograder/source/graderUtil.py", line 54, in wrapper
    result = func(*args, **kwargs)
File "/autograder/source/graderUtil.py", line 83, in wrapper
    result = func(*args, **kwargs)
File "/autograder/source/grader.py", line 23, in test_0
    submission.extractWordFeatures("a b a"))
File "/autograder/source/miniconda/envs/XCS221/lib/python3.6/unittest/case.py", line 829, in assertEqual
    assertion_func(first, second, msg=msg)
File "/autograder/source/miniconda/envs/XCS221/lib/python3.6/unittest/case.py", line 822, in _baseAssertEqual
    raise self.failureException(msg)
```

Just like in the local autograder, this error caused the test to fail.

Just like in the local autograder, start your debugging in line 23 of grader.py.

1a-1-hidden) Test multiple instances of the same word in a sentence. (0.0/3.0)

```
<class 'AssertionError': {'a': 23, 'ab': 22, 'aa': 24, 'c': 16, 'b': 15} != None
File "/autograder/source/miniconda/envs/XCS221/lib/python3.6/unittest/case.py", line 59, in testPartExecutor
    yield
File "/autograder/source/miniconda/envs/XCS221/lib/python3.6/unittest/case.py", line 605, in run
    testMethod()
File "/autograder/source/graderUtil.py", line 54, in wrapper
    result = func(*args, **kwargs)
File "/autograder/source/graderUtil.py", line 83, in wrapper
    result = func(*args, **kwargs)
File "/autograder/source/grader.py", line 31, in test_1
    self.compare_with_solution_or_wait(submission, 'extractWordFeatures', lambda f: f(sentence))
File "/autograder/source/graderUtil.py", line 183, in compare_with_solution_or_wait
    self.assertEqual(ans1, ans2)
File "/autograder/source/miniconda/envs/XCS221/lib/python3.6/unittest/case.py", line 829, in assertEqual
    assertion_func(first, second, msg=msg)
File "/autograder/source/miniconda/envs/XCS221/lib/python3.6/unittest/case.py", line 822, in _baseAssertEqual
    raise self.failureException(msg)
```

This error caused the test to fail.

Start your debugging in line 31 of grader.py.

Finally, here is what it looks like when basic and hidden tests pass in the remote autograder.

1a-0-basic) Basic test case. (2.0/2.0)

1a-1-hidden) Test multiple instances of the same word in a sentence. (3.0/3.0)

0 Introduction

In this assignment, we will begin implementing policy gradient methods on environments with discrete or continuous actions spaces. We will also explore variance reduction methods to allow us to improve upon the performance of the REINFORCE algorithm.

By the end of this assignment, you should have an understanding of how to implement some of the fundamental policy gradient algorithms. You should also be able to apply variance reduction techniques to improve upon the performance of REINFORCE and to derive/discuss variance reduction in policy gradient methods.

Advice

- It will take time to train your policy gradient implementation across all environments for various seeds. It is worthwhile budgeting at least 12 hours for all training jobs to run to completion.
- In this assignment, you will train your policy gradient implementation using MuJoCo (physics engine) to power the simulation environment. To learn more about MuJoCo please visit [MuJoCo Documentation](#).
- We provide conda environment for preparing development environment.
- To setup conda environment, for CPU environment use `environment.yml`, for CUDA use `environment_cuda.yml`. Here is the command:

```
$ conda update -n base conda
$ conda env create --file environment.yml # environment_cuda.yml for CUDA
```

```
bin/./lib/liblapack.3.dylib' (no such file), '/usr/local/lib/liblapack.3.dylib' (no such
file), '/usr/lib/liblapack.3.dylib' (no such file)
```

Coding Deliverables

For this assignment, please submit the following files to gradescope to receive points for coding questions:

- `src/submission/__init__.py`
- `src/submission/baseline_network.py`
- `src/submission/mlp.py`
- `src/submission/policy.py`
- `src/submission/policy_gradient.py`
- `src/submission/CartPole-v1-no-baseline-model-weights.pt`
- `src/submission/CartPole-v1-no-baseline-scores.npy`
- `src/submission/CartPole-v1-baseline-model-weights.pt`
- `src/submission/CartPole-v1-baseline-scores.npy`
- `src/submission/InvertedPendulum-v5-no-baseline-model-weights.pt`
- `src/submission/InvertedPendulum-v5-no-baseline-scores.npy`
- `src/submission/InvertedPendulum-v5-baseline-model-weights.pt`
- `src/submission/InvertedPendulum-v5-baseline-scores.npy`
- `src/submission/HalfCheetah-v5-no-baseline-model-weights.pt`
- `src/submission/HalfCheetah-v5-no-baseline-scores.npy`
- `src/submission/HalfCheetah-v5-baseline-model-weights.pt`
- `src/submission/HalfCheetah-v5-baseline-scores.npy`

1 Policy Gradient Methods

The goal of this problem is to experiment with policy gradient and its variants, including variance reduction methods. Your goal will be to set up policy gradient for both continuous and discrete environments, and implement a neural network baseline for variance reduction. The script for running the policy gradient algorithm is setup in `run.py`, and everything that you need to implement is in the files `baseline_network.py`, `mlp.py`, `policy.py` and `policy_gradient.py` within the submission folder. Each submission script has detailed instructions for each implementation task. We have also provided an overview of key steps in the algorithm below:

REINFORCE

Recall the policy gradient theorem:

$$\nabla_{\theta} J(\theta) = \mathbb{E}_{\pi_{\theta}} [\nabla_{\theta} \log \pi_{\theta}(a | s) Q^{\pi_{\theta}}(s, a)]$$

REINFORCE is a Monte Carlo policy gradient algorithm, so we will be using the sampled returns G_t as unbiased estimates of $Q^{\pi_{\theta}}(s, a)$. The REINFORCE estimator can be expressed as the gradient of the following objective function:

$$J(\theta) = \frac{1}{\sum T_i} \sum_{i=1}^{|D|} \sum_{t=1}^{T_i} \log(\pi_{\theta}(a_t^i | s_t^i)) G_t^i$$

where D is the set of all trajectories collected by policy π_{θ} , and $\tau^i = (s_0^i, a_0^i, r_0^i, s_1^i, \dots, s_{T_i}^i, a_{T_i}^i, r_{T_i}^i)$ is trajectory i .

Baseline

One difficulty of training with the REINFORCE algorithm is that the Monte Carlo sampled return(s) G_t can have high variance. To reduce variance, we subtract a baseline $b_{\phi}(s)$ from the estimated returns when computing the policy gradient. A good baseline is the state value function, $V^{\pi_{\theta}}(s)$, which requires a training update to ϕ to minimize the following mean-squared error loss:

$$L_{\text{MSE}}(\phi) = \frac{1}{\sum T_i} \sum_{i=1}^{|D|} \sum_{t=1}^{T_i} (G_t^i - b_{\phi}(s_t^i))^2$$

Advantage Normalization

After subtracting the baseline, we get the following new objective function:

$$J(\theta) = \frac{1}{\sum T_i} \sum_{i=1}^{|D|} \sum_{t=1}^{T_i} \log(\pi_{\theta}(a_t^i | s_t^i)) \hat{A}_t^i$$

where

$$\hat{A}_t^i = G_t^i - b_{\phi}(s_t^i)$$

A second variance reduction technique is to normalize the computed advantages, $\hat{A}_t^i$, so that they have mean 0 and standard deviation 1. From a theoretical perspective, we can consider centering the advantages to be simply adjusting the advantages by a constant baseline, which does not change the policy gradient. Likewise, rescaling the advantages effectively changes the learning rate by a factor of $1/\sigma$, where σ is the standard deviation of the empirical advantages.

Note: for the following coding questions some scripts contain member functions of different classes with the same name. In order to distinguish which function we refer to in each question we use the following syntax `class::function`.

(a) [2 points (Coding)]

Implement `build_mlp` within `submission/mlp.py` which will be used to construct a multi layer perceptron based on input argument values.

(b) [6 points (Coding)]

Implement the following functions within `submission/baseline_network.py` to create the baseline network for our policy gradient implementation:

- `BaselineNetwork::__init__`
- `BaselineNetwork::forward`
- `BaselineNetwork::calculate_advantage`
- `BaselineNetwork::update_baseline`

(c) [5 points (Coding)]

Implement `PolicyGradient::init_policy` within `policy_gradient.py` in order to initialize a network and optimizer for our implementation of policy gradient.

(d) [6 points (Coding)]

Implement `PolicyGradient::get_returns` within `policy_gradient.py` in order to calculate returns G_t given data about specific trajectories.

(e) [9 points (Coding)]

In this question, we will define the conditional probability distribution over actions for both discrete and continuous environments. Implement the following functions in `submission/policy.py` in order to define these distributions and how we sample from them:

- `BasePolicy::act`
- `CategoricalPolicy::action_distribution`
- `GaussianPolicy::__init__`
- `GaussianPolicy::std`
- `GaussianPolicy::action_distribution`

(f) [6 points (Coding)]

Implement the following functions in `submission/policy_gradient.py`:

- `PolicyGradient::update_policy`
- `PolicyGradient::normalize_advantage`

This will complete the `PolicyGradient` class by providing a network update rule as well as the option to normalize advantages we have calculated.

(g) [9 points (Coding)]

In this question, you will train your policy gradient implementation on three different environments.

- [CartPole-v1](#)
- [InvertedPendulum-v5](#)
- [HalfCheetah-v5](#)

For each of the 3 environments, choose 3 random seeds and run your policy gradient implementation both with and without a baseline.

Training Policy Gradient

The general form for running your policy gradient implementation is as follows (where `config_filename` is the name of a yaml file in the `src/config` folder with your training configuration):

```
$ python run.py --config_filename config_filename
```

Depending on the configuration file details you provide this command may train models for more than one seed which can take a while (especially for *HalfCheetah-v5*). As a result, you may wish to run these jobs as background processes in which case you may run the command as follows:

```
$ nohup python run.py --config_filename config_filename &
```

This will prevent the python process from ending when you close your terminal window as well as running the command in the background. Additionally, in the directory where you run the command a `nohup.out` file is created and contains the standard output from the process that you are running. Feel free to make use of this form of the command for the longer running processes.

Altering the Training Configuration

We have provided you with sample configuration files in the assignment `config` folder for you to use such as `config/cartpole_baseline.yml`. These configurations will be suitable to use directly in training your agent without altering the config file.

However, you may optionally alter the training configuration files directly to run your policy gradient implementation with different settings. Below we have briefly described some of the changes you could apply:

- To run your implementation with/without a baseline you may change the `use_baseline` option to either `true` or `false`.
- You may also choose to train multiple seeds under the one python process through specifying more than one seed in the list `seed` (equally you may specify a single seed in this list if you want to run just one).
- You may also wish to qualitatively observe the performance of your agent. To do so, you can record a single episode of the trained agent through changing the `record` option to `true` in the training configuration file for your run. Once your training run is complete you should see a video recording outputted in the `results` folder for the given run.

Results & Plotting

Once training is complete you should observe the creation of the following folder `src/results` which contains the results of your training runs. In addition, you should note that some files based on model evaluation have been created in the submission folder. This contains the weights and scores for one of your training runs for each environment and baseline/no baseline configuration (this will only populate once one of your training runs achieves evaluation scores above a certain threshold). You will need to upload these weights with your submission to receive full credit for this question. To plot your results for certain seeds run:

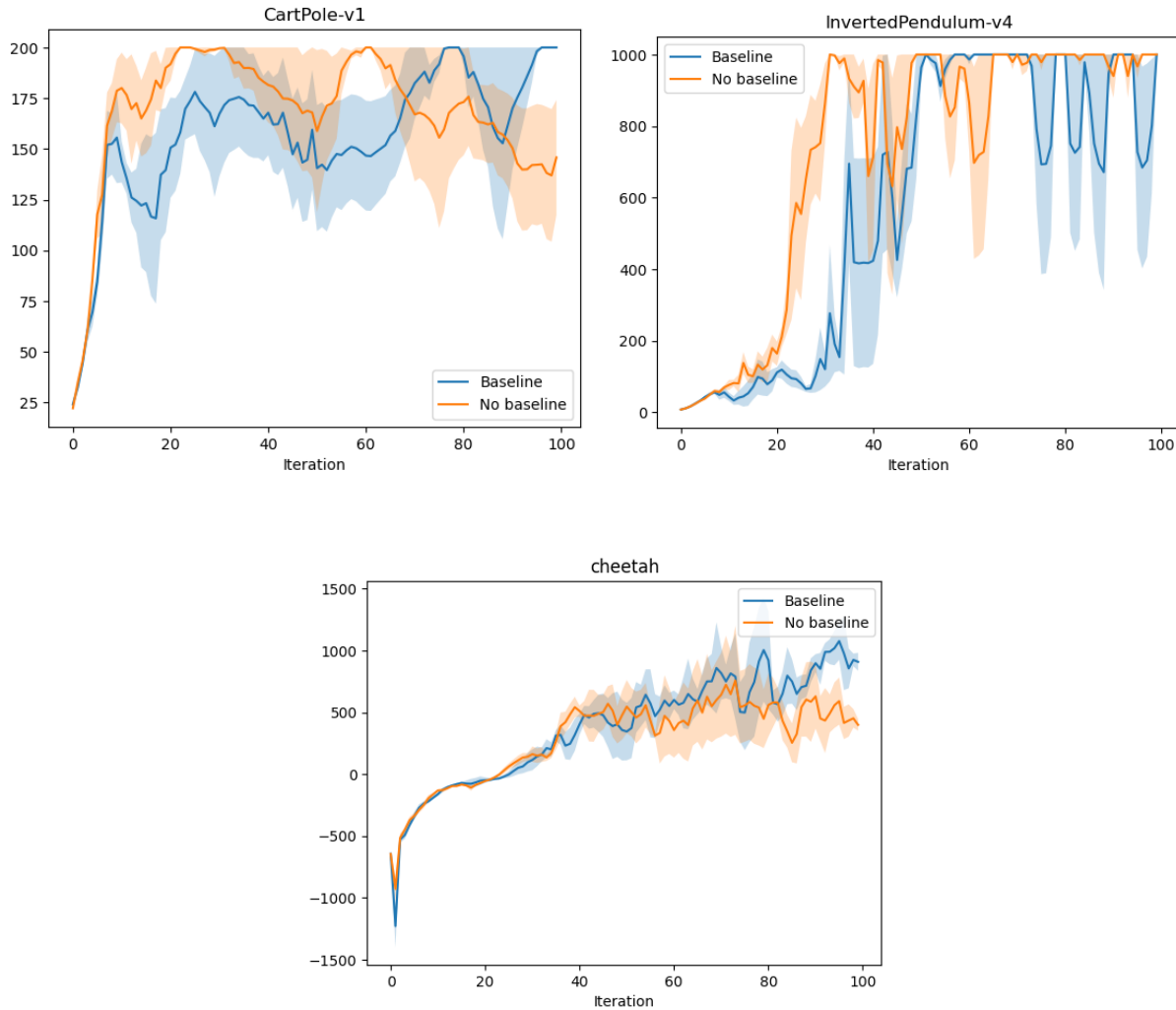
```
$ python run.py --plot_config_filename plot_config_filename
```

where `plot_config_filename` is the name of a yaml file in the `src/config` folder containing information on the seeds we wish to plot and for which environments. Please consult `config/plot.yml` for an example of the structure of this configuration file.

On the following page we provide sample outputs and an outline of the performance you can expect for the correct implementation.

Expected Results

We expect your plots to look similar to the plots which we have included:



The following performance benchmarks need to be achieved by your implementation in order to receive full credit for this question:

- CartPole-v1: agent should reach the maximum return of 200 with and without baseline (although it may not stay there)
- InvertedPendulum-v5: agent should reach the maximum return of 1000 with and without baseline (although it may not stay there)
- HalfCheetah-v5: agent should reach at least a score of 200 with and without baseline (in general it can achieve a much higher score e.g. 900)

2 Distributions induced by a policy

Suppose we have a single MDP and two policies for that MDP, π and π' . Naturally, we are often interested in the performance of policies obtained in the MDP, quantified by V^π and $V^{\pi'}$, respectively. If the reward function and transition dynamics of the underlying MDP are known to us, we can use standard methods for policy evaluation. There are many scenarios, however, where the underlying MDP model is not known and we must try to infer something about the performance of policy π' solely based on data obtained through executing policy π within the environment. In this problem, we will explore a classic result for quantifying the gap in performance between two policies that only requires access to data sampled from one of the policies.

Consider an infinite-horizon MDP $\mathcal{M} = \langle \mathcal{S}, \mathcal{A}, \mathcal{R}, \gamma \rangle$ and stochastic policies of the form $\pi : \mathcal{S} \rightarrow \Delta(\mathcal{A})^1$. Specifically, $\pi(a|s)$ refers to the probability of taking action a in state s , and $\sum_a \pi(a|s) = 1, \forall s$. For simplicity, we'll assume that this decision process has a single, fixed starting state $s_0 \in \mathcal{S}$.

(a) **[1 point (Written)]**

Consider a fixed stochastic policy and imagine running several rollouts of this policy within the environment. Naturally, depending on the stochasticity of the MDP $\mathcal{M}$ and the policy itself, some trajectories are more likely than others. Write down an expression for $\rho^\pi(\tau)$, the probability of sampling a trajectory $\tau = (s_0, a_0, s_1, a_1, \dots)$ from running π in $\mathcal{M}$. To put this distribution in context, recall that $V^\pi(s_0) = \mathbb{E}_{\tau \sim \rho^\pi} \left[\sum_{t=0}^{\infty} \gamma^t \mathcal{R}(s_t, a_t) \mid s_0 \right]$.

The probability of a specific trajectory is the product of the probabilities of the agent choosing an action based on its policy and the environment transitioning to the next state based on that action.

$$\rho^\pi(\tau) = p(s_0) \prod_{t=0}^{\infty} \pi(a_t|s_t) \mathcal{P}(s_{t+1}|s_t, a_t)$$

Because we assume that the MDP has a single, fixed starting state s_0 , i.e. $p(s_0) = 1$. Therefore,

$$\rho^\pi(\tau) = \prod_{t=0}^{\infty} \pi(a_t|s_t) \mathcal{P}(s_{t+1}|s_t, a_t)$$

(b) **[3 points (Written)]**

Just as ρ^π captures the distribution over trajectories induced by π , we can also examine the distribution over states induced by π . In particular, define the *discounted, stationary state distribution* of a policy π as

$$d^\pi(s) = (1 - \gamma) \sum_{t=0}^{\infty} \gamma^t p(s_t = s),$$

where $p(s_t = s)$ denotes the probability of being in state s at timestep t while following policy π ; your answer to the previous part should help you reason about how you might compute this value. Consider an arbitrary function $f : \mathcal{S} \times \mathcal{A} \rightarrow \mathbb{R}$. Prove the following identity:

$$\mathbb{E}_{\tau \sim \rho^\pi} \left[\sum_{t=0}^{\infty} \gamma^t f(s_t, a_t) \right] = \frac{1}{(1 - \gamma)} \mathbb{E}_{s \sim d^\pi} \left[\mathbb{E}_{a \sim \pi(s)} [f(s, a)] \right].$$

Hint: You may find it helpful to first consider how things work out for $f(s, a) = 1, \forall (s, a) \in \mathcal{S} \times \mathcal{A}$.

Hint: What is $p(s_t = s)$?

Firstly, apply the linearity of expectation and then expand the expectation.

¹For a finite set $\mathcal{X}$, $\Delta(\mathcal{X})$ refers to the set of categorical distributions with support on $\mathcal{X}$ or, equivalently, the $\Delta^{|\mathcal{X}|-1}$ probability simplex.

$$\begin{aligned}
\mathbb{E}_{\tau \sim \rho^\pi} \left[\sum_{t=0}^{\infty} \gamma^t f(s_t, a_t) \right] &= \sum_{t=0}^{\infty} \gamma^t \mathbb{E}_{\tau \sim \rho^\pi} [f(s_t, a_t)] \\
&= \sum_{t=0}^{\infty} \gamma^t \sum_{s \in \mathcal{S}} \sum_{a \in \mathcal{A}} p(s_t = s, a_t = a) f(s, a) \\
&= \sum_{t=0}^{\infty} \gamma^t \sum_{s \in \mathcal{S}} \sum_{a \in \mathcal{A}} p(s_t = s) p(a_t = a | s_t = s) f(s, a) \\
&= \sum_{t=0}^{\infty} \gamma^t \sum_{s \in \mathcal{S}} \sum_{a \in \mathcal{A}} p(s_t = s) \pi(a | s) f(s, a) \\
&= \sum_{s \in \mathcal{S}} \sum_{a \in \mathcal{A}} \pi(a | s) f(s, a) \left(\sum_{t=0}^{\infty} \gamma^t p(s_t = s) \right)
\end{aligned}$$

We have the definition of the discounted, stationary state distribution

$$d^\pi(s) = (1 - \gamma) \sum_{t=0}^{\infty} \gamma^t p(s_t = s)$$

Rearrange the equation we can get solve $\sum_{t=0}^{\infty} \gamma^t p(s_t = s)$

$$\sum_{t=0}^{\infty} \gamma^t p(s_t = s) = \frac{d^\pi(s)}{1 - \gamma}$$

Substitute back and we get

$$\begin{aligned}
\mathbb{E}_{\tau \sim \rho^\pi} \left[\sum_{t=0}^{\infty} \gamma^t f(s_t, a_t) \right] &= \sum_{s \in \mathcal{S}} \sum_{a \in \mathcal{A}} \pi(a | s) f(s, a) \left(\frac{d^\pi(s)}{1 - \gamma} \right) \\
&= \frac{1}{1 - \gamma} \sum_{s \in \mathcal{S}} d^\pi(s) \left(\sum_{a \in \mathcal{A}} \pi(a | s) f(s, a) \right) \\
&= \frac{1}{1 - \gamma} \mathbb{E}_{s \sim d^\pi} [\mathbb{E}_{a \sim \pi(s)} [f(s, a)]]
\end{aligned}$$

(c) [5 points (Written)] For any policy π , we define the following function

$$A^\pi(s, a) = Q^\pi(s, a) - V^\pi(s).$$

$A^\pi(s, a)$ is known as the advantage function and shows up in a lot of policy gradient based RL algorithms, which we shall see later in the class. Intuitively, it is the additional benefit one gets from first following action a and then following π , instead of always following π . Prove that the following statement holds for all policies π, π' :

$$V^\pi(s_0) - V^{\pi'}(s_0) = \frac{1}{(1 - \gamma)} \mathbb{E}_{s \sim d^\pi} \left[\mathbb{E}_{a \sim \pi(s)} [A^{\pi'}(s, a)] \right].$$

Hint 1: Try adding and subtracting a term that will let you bring $A^{\pi'}(s, a)$ into the equation. What happens on adding and subtracting $\sum_{t=0}^{\infty} \gamma^{t+1} V^{\pi'}(s_{t+1})$ on the LHS?

Hint 2: Recall the [tower property of expectation](#) which says that $\mathbb{E}[X] = \mathbb{E}[\mathbb{E}[X | Y]]$.

After proving this result, you might already begin to appreciate why this represents a useful theoretical contribution. We are often interested in being able to control the gap between two value functions and this result provides a new mechanism for doing exactly that, when the value functions in question belong to two particular policies of the MDP.

Additionally, to see how this result is of practical importance as well, suppose the data-generating policy in the above identity π is some current policy we have in hand and π' represents some next policy we would like to optimize for; concretely, this scenario happens quite often when π is a neural network and π' denotes the same network with updated parameters. As is often the case with function approximation, there are sources of instability and, sometimes, even small parameter updates can lead to drastic changes in policy performance, potentially degrading (instead of improving) the performance of the current policy π . These realities of deep learning motivate a desire to occasionally be conservative in our updates and attempt to reach a new policy π' that provides only a modest improvement over π . Practical approaches can leverage the above identity to strike the right balance between making progress and maintaining stability.

$$\begin{aligned}
V^\pi(s_0) - V^{\pi'}(s_0) &= \mathbb{E}_{\tau \sim \rho^\pi} \left[\sum_{t=0}^{\infty} \gamma^t \mathcal{R}(s_t, a_t) \right] - V^{\pi'}(s_0) \\
&= \mathbb{E}_{\tau \sim \rho^\pi} \left[\sum_{t=0}^{\infty} \gamma^t \left(\mathcal{R}(s_t, a_t) + V^{\pi'}(s_t) - V^{\pi'}(s_t) \right) \right] - V^{\pi'}(s_0) \\
&= \mathbb{E}_{\tau \sim \rho^\pi} \left[\sum_{t=0}^{\infty} \gamma^t \left(\mathcal{R}(s_t, a_t) + \gamma V^{\pi'}(s_{t+1}) - V^{\pi'}(s_t) \right) \right] \\
&= \mathbb{E}_{\tau \sim \rho^\pi} \left[\sum_{t=0}^{\infty} \gamma^t \left(Q^{\pi'}(s_t, a_t) - V^{\pi'}(s_t) \right) \right] && \text{Definition of } Q^{\pi'}(s_t, a_t) \\
&= \mathbb{E}_{\tau \sim \rho^\pi} \left[\sum_{t=0}^{\infty} \gamma^t A^{\pi'}(s_t, a_t) \right] && \text{Definition of } A^{\pi'}(s_t, a_t) \\
&= \frac{1}{(1-\gamma)} \mathbb{E}_{s \sim d^\pi} [\mathbb{E}_{a \sim \pi(s)} [A^{\pi'}(s, a)]] && \text{From 2.b}
\end{aligned}$$

3 Ethical concerns with Policy Gradients

In this assignment, we focus on policy gradients, an extremely popular and useful model-free technique for RL. However, policy gradients collect data from the environment with a potentially suboptimal policy during the learning process. While this is acceptable in simulators like Mujoco or Atari, such exploration in real world settings such as healthcare and education presents challenges.

Consider a case study of a Stanford CS course considering introducing a RL-based chat bot for office hours. For each assignment, some students will be given 100% human CA office hours; others 100% chatbot; others a mix of both. The reward signal is the student grades on each assignment. Since the AI chatbot will learn through experience, at any given point in the quarter, the help given by the chatbot might be better or worse than the help given by a randomly selected human CA.

If each time students are randomly assigned to each condition, some students will be assigned more chatbot hours and others fewer. In addition, some students will be assigned more chatbot hours at the beginning of the term (when the chatbot has had fewer interactions and may have lower effectiveness) and fewer at the end, and vice versa. All students will be graded according to the same standards, regardless of which type of help they have received.

Researchers who experiment on human subjects are morally responsible for ensuring their well being and protecting them from being harmed by the study. A foundational document in research ethics, the [Belmont Report](#), identifies three core principles of responsible research:

1. **Respect for persons:** individuals are capable of making choices about their own lives on the basis of their personal goals. Research participants should be informed about the study they are considering undergoing, asked for their consent, and not coerced into giving it. Individuals who are less capable of giving informed consent, such as young children, should be protected in other ways.
2. **Beneficence:** the principle of beneficence describes an obligation to ensure the well-being of subjects. It has been summarized as “do not harm” or “maximize possible benefits and minimize possible harms.”
3. **Justice:** the principle of justice requires treating all people equally and distributing benefits and harms to them equitably.

(a) [3 points (Written)]

In 4-6 sentences, describe **two** experimental design or research choices that researchers planning the above experiment ought to make in order to respect these principles. Justify the importance of these choices using one of the three ethical principles above and indicating which principle you have chosen. For example, “Researchers ought to ensure that students advised by the chatbot are able to revise their assignments after submission with the benefit of human advice if needed. If they did not take this precaution, the principle of justice would be violated because the risk of harm from poor advice from the AI chatbot would be distributed unevenly.”

1. To follow **Respect for persons**, the researcher must properly inform the students about the study's purpose, the method they will implement including the use of AI chatbot and random assignment and any other possible risk with the student's consent.
2. To minimize possible harms (as stated in **Beneficence**), the researcher should ensure that the students can withdraw from the study at any time without penalty if they feel the chatbot is ineffective.

This handout includes space for every question that requires a written response. Please feel free to use it to handwrite your solutions (legibly, please). If you choose to typeset your solutions, the —README.md— for this assignment includes instructions to regenerate this handout with your typeset L^AT_EX solutions.

2.a

The probability of a specific trajectory is the product of the probabilities of the agent choosing an action based on its policy and the environment transitioning to the next state based on that action.

$$\rho^\pi(\tau) = p(s_0) \prod_{t=0}^{\infty} \pi(a_t|s_t) \mathcal{P}(s_{t+1}|s_t, a_t)$$

Because we assume that the MDP has a single, fixed starting state s_0 , i.e. $p(s_0) = 1$. Therefore,

$$\rho^\pi(\tau) = \prod_{t=0}^{\infty} \pi(a_t|s_t) \mathcal{P}(s_{t+1}|s_t, a_t)$$

2.b

Firstly, apply the linearity of expectation and then expand the expectation.

$$\begin{aligned}
 \mathbb{E}_{\tau \sim \rho^\pi} \left[\sum_{t=0}^{\infty} \gamma^t f(s_t, a_t) \right] &= \sum_{t=0}^{\infty} \gamma^t \mathbb{E}_{\tau \sim \rho^\pi} [f(s_t, a_t)] \\
 &= \sum_{t=0}^{\infty} \gamma^t \sum_{s \in \mathcal{S}} \sum_{a \in \mathcal{A}} p(s_t = s, a_t = a) f(s, a) \\
 &= \sum_{t=0}^{\infty} \gamma^t \sum_{s \in \mathcal{S}} \sum_{a \in \mathcal{A}} p(s_t = s) p(a_t = a | s_t = s) f(s, a) \\
 &= \sum_{t=0}^{\infty} \gamma^t \sum_{s \in \mathcal{S}} \sum_{a \in \mathcal{A}} p(s_t = s) \pi(a|s) f(s, a) \\
 &= \sum_{s \in \mathcal{S}} \sum_{a \in \mathcal{A}} \pi(a|s) f(s, a) \left(\sum_{t=0}^{\infty} \gamma^t p(s_t = s) \right)
 \end{aligned}$$

We have the definition of the discounted, stationary state distribution

$$d^\pi(s) = (1 - \gamma) \sum_{t=0}^{\infty} \gamma^t p(s_t = s)$$

Rearrange the equation we can get solve $\sum_{t=0}^{\infty} \gamma^t p(s_t = s)$

$$\sum_{t=0}^{\infty} \gamma^t p(s_t = s) = \frac{d^\pi(s)}{1 - \gamma}$$

Substitute back and we get

$$\begin{aligned}
 \mathbb{E}_{\tau \sim \rho^\pi} \left[\sum_{t=0}^{\infty} \gamma^t f(s_t, a_t) \right] &= \sum_{s \in \mathcal{S}} \sum_{a \in \mathcal{A}} \pi(a|s) f(s, a) \left(\frac{d^\pi(s)}{1 - \gamma} \right) \\
 &= \frac{1}{1 - \gamma} \sum_{s \in \mathcal{S}} d^\pi(s) \left(\sum_{a \in \mathcal{A}} \pi(a|s) f(s, a) \right) \\
 &= \frac{1}{1 - \gamma} \mathbb{E}_{s \sim d^\pi} [\mathbb{E}_{a \sim \pi(s)} [f(s, a)]]
 \end{aligned}$$

2.c

$$\begin{aligned}
V^\pi(s_0) - V^{\pi'}(s_0) &= \mathbb{E}_{\tau \sim \rho^\pi} \left[\sum_{t=0}^{\infty} \gamma^t \mathcal{R}(s_t, a_t) \right] - V^{\pi'}(s_0) \\
&= \mathbb{E}_{\tau \sim \rho^\pi} \left[\sum_{t=0}^{\infty} \gamma^t \left(\mathcal{R}(s_t, a_t) + V^{\pi'}(s_t) - V^{\pi'}(s_t) \right) \right] - V^{\pi'}(s_0) \\
&= \mathbb{E}_{\tau \sim \rho^\pi} \left[\sum_{t=0}^{\infty} \gamma^t \left(\mathcal{R}(s_t, a_t) + \gamma V^{\pi'}(s_{t+1}) - V^{\pi'}(s_t) \right) \right] \\
&= \mathbb{E}_{\tau \sim \rho^\pi} \left[\sum_{t=0}^{\infty} \gamma^t \left(Q^{\pi'}(s_t, a_t) - V^{\pi'}(s_t) \right) \right] && \text{Definition of } Q^{\pi'}(s_t, a_t) \\
&= \mathbb{E}_{\tau \sim \rho^\pi} \left[\sum_{t=0}^{\infty} \gamma^t A^{\pi'}(s_t, a_t) \right] && \text{Definition of } A^{\pi'}(s_t, a_t) \\
&= \frac{1}{(1-\gamma)} \mathbb{E}_{s \sim d^\pi} [\mathbb{E}_{a \sim \pi(s)} [A^{\pi'}(s, a)]] && \text{From 2.b}
\end{aligned}$$

3.a

1. To follow **Respect for persons**, the researcher must properly inform the students about the study's purpose, the method they will implement including the use of AI chatbot and random assignment and any other possible risk with the student's consent.
2. To minimize possible harms (as stated in **Beneficence**), the researcher should ensure that the students can withdraw from the study at any time without penalty if they feel the chatbot is ineffective.